

Session 0

Python for

Data Science

Everything you need to start Lab 1. Nothing you do not.

Before Week 1 • two hours • Colab only

Course page • sabokrou.github.io/teaching/data-science-fundamentals

IF YOU HAVE NEVER PROGRAMMED

This session is for you.

No installation. No terminal. No prior experience assumed. You will type every line yourself — that is the point.

Course page

all slides, labs,
book and data



Two hours from nothing to reading a real dataset

0–10	What Colab is, and running your first cell
10–30	Values, variables, and the four types you will meet
30–45	Lists, and asking true-or-false questions
45–60	if / else, and for loops
60–70	Break
70–85	Functions, and how to read an error message
85–110	pandas: loading a table and asking it questions
110–120	The seven lines for Lab 1, and how you are assessed

By the end you can

- open a notebook and run code in it
- store a value, change it, and print it
- write a function that returns something
- load a CSV file and say how many rows it has
- read an error message instead of panicking

★ GROUND RULE

Type every line. Do not copy and paste, and do not just watch. Typing is how the syntax gets into your hands.

And why this course is not taught in C++ or Java

Python, for this course

The whole textbook is written in it, and the dataset tools — pandas, matplotlib, scikit-learn — are Python libraries.

It runs in the browser through Colab, so nothing is installed and nothing breaks on your machine.

You can load a CSV and compute an average in three lines. In this course you are learning to think about data, and the language should stay out of the way.

C++ is a good language — for other work

It is faster, and it teaches you how memory actually works. If you go on to systems, graphics or embedded work, you will need it.

But for data analysis it costs you a great deal: a compiler to install, far more code for the same result, and no equivalent of pandas that a beginner can use in week one.

You would spend the term fighting the language instead of learning the subject.

WORTH SAYING PLAINLY

Learning Python here does not close the door on C++. Languages are much easier to learn once you have one.

Colab is a notebook: a page made of cells

A code cell

holds Python. Press Shift + Enter to run it. The result appears underneath.

A text cell

holds writing. This is where you explain what you did and what it means.

The runtime

is the computer running your code, in Google's data centre. It remembers what you ran.

```
print("Hello")  
Hello
```

Runtime → Restart and run all

Runs every cell again, from empty, in order.

If it works after that, your notebook is honest. If it breaks, it was only ever working by accident.

You will do this before every submission, all term.

★ DO IT NOW

Open a new notebook. Type `print("Hello")` and run it. Everyone, before we go on.

A variable is a name for a value

```
students = 1200
course_name = "Fundamentals of Data Science"

print(students)
1200
```

Read = as “becomes”

`students = 1200` does not state a fact. It is an instruction: make the name `students` refer to 1200.

So `students = students + 1` is not nonsense. It means: take what `students` is now, add one, put it back.

Naming rules

Letters, digits and underscores. No spaces. Cannot start with a digit. Case matters: `Students` and `students` are different names.

Naming habits

Use names that say what the thing is. `entry_score`, not `x`. You will read your own code in six weeks.

★ TRY IT

Make a variable called `my_name` holding your name, then print it.

Python treats these differently, and so should you

int

a whole number

`logins = 14`**float**

a number with a decimal part

`score = 72.5`**str**

text, always in quotes

`mode = "Part-time"`**bool**

True or False, no quotes

`passed = True`

Why it matters here

"14" is text. 14 is a number.

"14" + "1" gives "141".

14 + 1 gives 15.

When a student ID is read as a number instead of text, the leading zeros vanish and it stops matching the other table. You will meet exactly this in Week 1.

★ CHECK

Run `type(72.5)` and `type("72.5")`. What comes back, and why is the difference not trivial?

A list holds values in order

```
scores = [72, 65, 58, 91]

print(len(scores))      # how many
4
print(scores[0])        # the first one
72
print(sum(scores) / len(scores))
71.5
```

Counting starts at zero

scores[0] is the first item. scores[1] is the second.

scores[4] does not exist and will raise IndexError.

Everyone finds this annoying for about a week, and then never thinks about it again.

A column of a dataset is very like a list: many values of the same kind, in order, one per row.

TRY IT

Make a list of four numbers and print the average. Then print the last one.

Comparisons produce a bool

```
score = 72

print(score > 50)
True
print(score == 100)    # == asks, = tells
False
```

==

is equal to

!=

is not equal to

> <

greater, less than

>= <=

or equal to

and

both must be true

or

either will do

One equals sign sets a value. Two equals signs ask a question. Writing = where you meant == is the most common beginner error in any language.

★ PREDICT, THEN RUN

What does `72 > 50` and `72 > 100` give? Say it out loud before you press run.

if, elif, else

```
score = 72

if score >= 70:
    print("Distinction")
elif score >= 50:
    print("Pass")
else:
    print("Fail")

Distinction
```

The colon and the indent

The colon says “what follows belongs to this.”

The four spaces of indentation are not decoration — they are how Python knows which lines are inside the if.

Get them wrong and you get `IndentationError`, which is Python being helpful, not hostile.

TRY IT

Change score to 45 and run it again. Then to 50. Does 50 pass?

A for loop walks through a list

```
scores = [72, 65, 58, 91]

for s in scores:
    print(s)

72
65
58
91
```

Read it as a sentence

“For each item in scores, call it s, and do the indented part.”

The name s is yours to choose. for score in scores: reads better and does exactly the same thing.

In this course you will write very few loops. pandas does the walking for you — but you need to recognise one when you read it.

TRY IT

Write a loop that prints each score doubled.

SHORT BREAK — TEN MINUTES

After the break: functions, errors, and your first real dataset

If something has not worked yet, come and find me now
rather than at the end.

SO FAR YOU CAN

store a value
choose between two paths
repeat something
ask a true-or-false question

That is most of what any program does.

A function takes something in and gives something back

```
def average(numbers):  
    return sum(numbers) / len(numbers)  
  
scores = [72, 65, 58, 91]  
print(average(scores))  
71.5
```

Three parts

`def average(...)` names it.

`numbers` is the input — whatever you hand it when you call it.

`return` hands a value back. A function with no return gives you `None`, which is the commonest cause of “why is my answer empty?”

★ TRY IT — THIS ONE IS ASSESSED IN
WEEK 1

Write a function that takes a list and returns the largest value in it.

An error message is information, not a verdict

```
print(studnets)

NameError: name 'studnets' is not defined
```

Nothing is broken

An error means Python stopped before doing something wrong. That is the good outcome.

The dangerous case is code that runs and gives a wrong answer.

Read the last line first. It names the problem. The lines above it tell you where.

NameError a name you used does not exist — usually a typo, or a cell you did not run

TypeError you did something to a value its kind does not allow, e.g. "5" + 5

IndentationError your spacing does not match what the colon promised

FileNotFoundError the path is wrong, or the file is not where you think it is

★ BREAK IT ON PURPOSE

Misspell a variable name and run it. Read the message. Fix it. Do that twice.

Other people have already written most of what you need

```
import pandas as pd

# pd is now a short name for the whole library
```

Already installed

In Colab, pandas is there. You import it, you do not install it.

What pandas gives you

A DataFrame: a table, in Python, that you can ask questions of.

Think of it as a spreadsheet you talk to in code rather than click on.

Why as pd

Convention, nothing more. Every book, every answer online, and every line of your textbook writes pd.

Follow it, so that other people's code reads like yours.

RUN IT

import pandas as pd — nothing visible happens. That is success.

Three lines, and you have real data

```
import pandas as pd
students = pd.read_csv("students.csv")

print(students.shape)
(1200, 7)

students.head(3)
shows the first three rows
```

shape is (rows, columns)

1,200 rows and 7 columns.

This is the first thing to print for any file you open, every time, before anything else.

If the number of rows is not what you expected, stop and find out why. That instinct is the whole of Week 1.

★ TRY IT

Load students.csv and print its shape. Then look at the first five rows.

Columns, filters, and summaries

```
students["entry_score"]          # one column

students["entry_score"].mean()    # its average
65.86

students["programme"].value_counts()
how many students in each programme

students["student_id"].nunique()  # distinct values
```

Whole column at once

`.mean()` averages every row without a loop. So do `.sum()`, `.min()`, `.max()`.

`value_counts()` is the one you will use most: it answers “how many of each?”

`nunique()` counts distinct values — remember this one. It is the line that separates 4,665 registrations from 1,200 students in Week 1.

★ TRY IT

How many students are in each `study_mode`? Which column would tell you how many distinct programmes exist?

Filtering rows, and summarising by group

```
high = students[students["entry_score"] > 70]
print(high.shape)
```

Keep only the rows where the condition is True.

```
students.groupby("programme")
    ["entry_score"].mean()
```

The filter is just a comparison

`students["entry_score"] > 70` gives True or False for every row. The outer brackets keep the True ones.

groupby: split, then summarise

Split the rows by programme, then take the mean of `entry_score` within each one.

One line, one answer per group. You will use this in almost every lab.

★ TRY IT

What is the mean `entry_score` for each `study_mode`? One line.

Seven lines. If you can write these, you are ready

```
import pandas as pd

students = pd.read_csv("students.csv")

print(students.shape)
students.head()
print(students["programme"].value_counts())
print(students["student_id"].nunique())
print(students["entry_score"].mean())
```

That is the whole toolkit

Load it. Check its size. Look at real rows. Count the categories. Count the distinct values. Average a number.

Everything in Lab 1 is one of these six moves, applied to four files.

Screenshot this slide.

★ BEFORE YOU LEAVE

Run all seven lines, in a fresh notebook, in order. Then Runtime → Restart and run all.

Every one of these will happen to you

Editing a cell and not running it

Nothing changes. Press Shift + Enter.

Using = where you meant ==

One sets, two asks.

Forgetting quotes around text

NameError, because Python looks for a variable of that name.

Running cells out of order

Works for you, fails for everyone else. Restart and run all.

Copying code you have not read

It runs, you cannot fix it, and you cannot explain it.

★ THE RULE THAT PREVENTS MOST OF THEM

Runtime → Restart and run all, before you submit anything. All term.

Where your mark comes from, and what you hand in

Midterm & final exams	45%	Written. Applied questions about tables, results, code and charts — never definitions.
Weekly lab notebooks	15%	One notebook a week, in the LMS. Pass / revise / fail. Best 9 of 12.
Data Quality Report	10%	Week 4. Find what is wrong with the data, and say what you did about it.
EDA & Visualization Brief	10%	Week 7. Charts that each make a point, with the claim type named.
Modelling Assignment	10%	Week 11. A model, plus why you set the threshold where you did.
Capstone + dashboard	10%	Week 12. Your own dataset, start to finish, limitations included.

★ THE RULE BEHIND ALL OF IT

Every submission from Week 4 must run top to bottom after Restart and run all. Today you learned how.

You now know enough Python

Not all of Python. Enough of it — and the rest arrives when you need it, one week at a time.

Do this week

Finish Lab 0: data in Drive at
MyDrive/fds/data/, setup cell returning the
four shapes.

Practise

Reopen your notebook and run the seven
lines on a different file. enrolments.csv, say.

Bring to Week 1

Nothing but a pen. Week 1 is about thinking,
not typing.

If something did not work today, tell me before you leave. It will not fix itself, and Lab 1 assumes it works.